EXPLAINABLE - DRIFT-AWARE - COST-ACCOUNTED

TabML-Guard Report — Appliances Energy Prediction

data/appliances_energy.csv · 19,735 rows × 27 features · regression

RECOMMENDED MODEL

linear

RMSE (HELD-OUT)

94.1187

AT A GLANCE

- Simple model wins
- Tuning: real gain
- Inputs drifted, performance held

Headline finding

linear is the recommended model, scoring 94.1187 rmse on a strictly held-out, time-ordered test set.

Recommendation

Tuning improved the score by 13.1% over defaults (best strategy: bayes). At an equal budget Bayesian search edged out random search, but cost 4.3x the wall-clock. The input distribution shifted materially (PSI above 0.25) while accuracy held up - monitor, but no retrain is urgent yet.

On model complexity

A regularized linear baseline matched or beat every ensemble here. That is a finding, not a failure: it says the signal in this dataset is largely additive, and it argues for shipping the cheaper, more interpretable model.

This report was produced automatically by the TabML-Guard pipeline. Every number below comes from a single run on a strictly held-out, time-ordered test split - no test data was used for model selection or tuning.

CONTENTS

Table of contents

Dataset, profile and configuration	3
Regularized linear baselines	3
Ensemble and boosting models	3
Hyperparameter optimization: benefit vs cost	4
Explainability and prediction reliability	4
Concept drift and statistical significance	6
Appendix A - Scope and limitations	7
Appendix B - Glossary	8

STAGE 0

Dataset, profile and configuration

The pipeline begins by profiling the raw table and deciding, from the target column alone, whether this is a classification or a regression problem. When a time column is present the split is chronological - the model trains on the past and is scored on the future.

Rows	19,735
Features	27
Target column	Appliances
Detected task	regression
Missing values	0
Time column	date
Time span	2016-01-11 17:00:00 -> 2016-05-27 18:00:00
Target statistics	min=10.0000 max=1,080 mean=97.6950 std=102.5249

Configuration used

This is the entire dataset-specific input to the run. Everything else is generic pipeline code.

```
dataset.source        csv
dataset.name          -
dataset.path          data/appliances_energy.csv
dataset.target        Appliances
dataset.time_col      date
dataset.drop_cols     []
split                 train=0.6  test=0.4
models.boosters       ['xgboost', 'lightgbm', 'catboost', 'random_forest']
models.stacking       True
tuning.budget_trials  40
tuning.cv_splits      3
drift.n_chunks        10
drift.retrain         sliding
random_state          42
```

STAGE 1

Regularized linear baselines

Before reaching for an ensemble, the pipeline establishes what a well-regularized linear model can do. This is the honesty check: if a linear model is competitive, the extra complexity of boosting has to justify itself.

model	rmse	mae	r2	fit
linear	**94.1187**	**59.5023**	**0.0648**	**0.1s**
lasso	94.1334	59.5413	0.0645	19.4s
ridge	94.1602	59.6030	0.0640	0.0s
elasticnet	94.3458	59.9957	0.0603	14.1s

the 4 regularizers span only 0.2271 rmse (best 'linear' 94.1187) — with abundant rows and few features, penalties barely move the fit at the default strength; Lasso path: at alpha=10 only 5/27 features survive (sparsity)

STAGE 2

Ensemble and boosting models

Gradient-boosted trees plus a stacking meta-model, all scored through the same evaluation path as the baselines so the leaderboard rows are directly comparable.

model	rmse	mae	r2	fit
linear	94.1187	59.5023	0.0648	0.1s
lasso	94.1334	59.5413	0.0645	19.4s
ridge	94.1602	59.6030	0.0640	0.0s
elasticnet	94.3458	59.9957	0.0603	14.1s
catboost	**118.6000**	**89.4261**	**-0.4850**	**1.1s**
lightgbm	147.0279	115.8324	-1.2822	0.3s
random_forest	163.0187	139.9893	-1.8056	4.0s
stacking	198.9678	167.1783	-3.1795	17.9s
xgboost	222.7032	184.3076	-4.2361	0.5s

Best ensemble: **catboost**. The strongest single booster (**catboost**) is the one carried into the tuning comparison in Stage 3.

STAGE 3

Hyperparameter optimization: benefit vs cost

Four strategies compete on **catboost**: library defaults, an exhaustive coarse grid, random search, and Bayesian optimization (Optuna TPE). Random and Bayesian are given an **equal budget** of 40 trials, which makes them a fair head-to-head. Crucially, the table reports not just the score but what each strategy *cost*.

strategy	test rmse	wall clock	model fits	cv score
default	111.6041	2.8s	1	-
grid	103.6200	22.8s	54	-98.4337
random	100.6344	193.9s	120	-97.8578
bayes	**96.9694**	**838.5s**	**120**	**-96.9231**

The *cv score* column is the internal cross-validated score each search optimized (scikit-learn's convention is higher-is-better, so error metrics appear negated). The *test* column is the winning configuration refit once and scored on the untouched held-out period - that is the number to trust.

Is tuning worth it?

best tuner 'bayes' reached rmse=96.9694 vs default 111.6041 (+14.6347); at equal budget, Bayesian search won (rmse: bayes 96.9694 @ 838.45s vs random 100.6344 @ 193.87s)

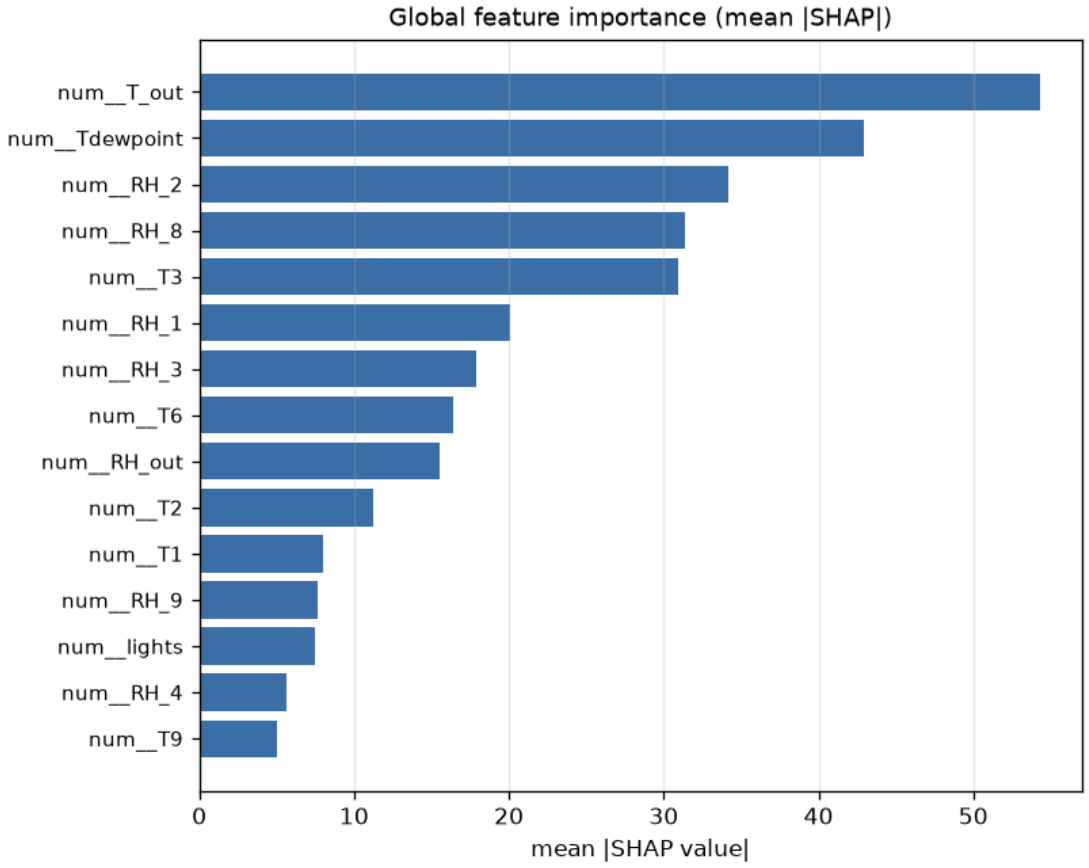
STAGE 4

Explainability and prediction reliability

SHAP attributions for **linear**, using the explainer matched to the model family (tree, linear, or model-agnostic fallback). Global importance answers *what drives this model overall*; the local example shows how a single prediction was

reached.

feature	mean |SHAP|
num__T_out	54.28369
num__Tdewpoint	42.96265
num__RH_2	34.14321
num__RH_8	31.37328
num__T3	30.89721
num__RH_1	20.04009
num__RH_3	17.85155
num__T6	16.40537
num__RH_out	15.55756
num__T2	11.22674
num__T1	7.96479
num__RH_9	7.63515
num__lights	7.43323
num__RH_4	5.59117
num__T9	5.05703



Global feature importance (mean |SHAP|)

reliability analysis unavailable.

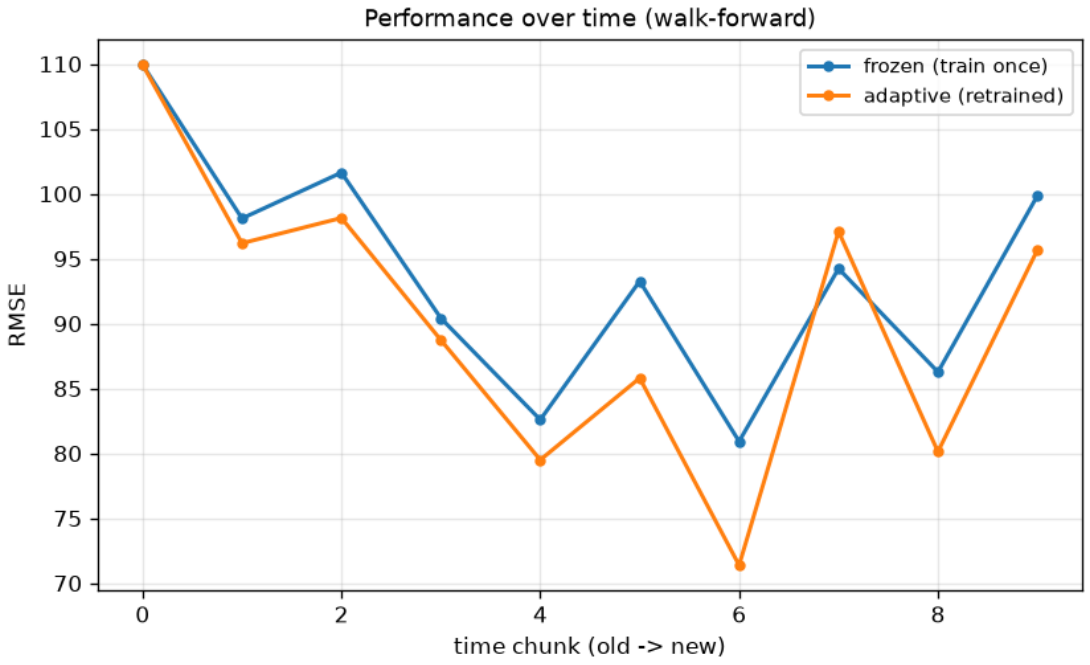
STAGE 5

Concept drift and statistical significance

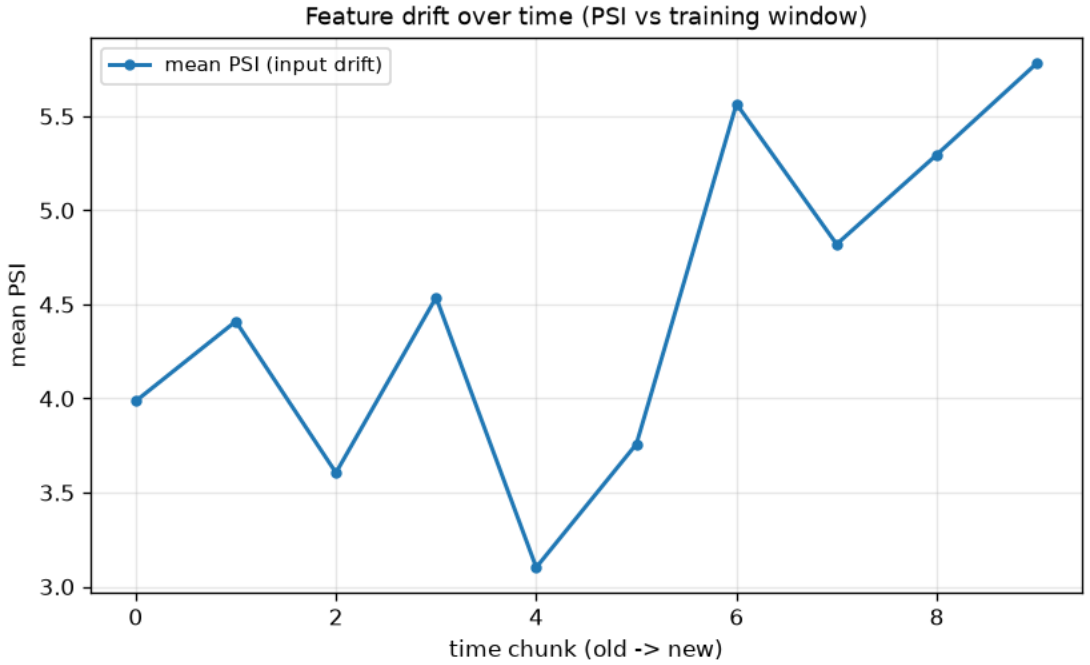
The held-out period is replayed in 10 chronological chunks. A **frozen** model (trained once, never updated) is compared against an **adaptive** model that predicts each chunk *before* learning from it - so there is no look-ahead. Retraining mode: **sliding**.

chunk	rows	frozen	adaptive	mean PSI
0	790	109.9472	109.9472	3.986
1	790	98.0982	96.1889	4.410
2	790	101.6319	98.1372	3.605
3	790	90.4172	88.7326	4.537
4	789	82.5920	79.4962	3.102
5	789	93.2671	85.8021	3.757
6	789	80.9215	71.3850	5.566
7	789	94.2381	97.0984	4.820
8	789	86.2698	80.1312	5.296
9	789	99.8473	95.6657	5.782

Chunk 1 is a built-in consistency check: both models are identical there by construction, since neither has seen the chunk yet.



Frozen vs adaptive performance across the held-out period



Input drift (PSI) against the training window

Mean score, frozen model	`93.7230`
Mean score, adaptive model	`90.2585`
First third -> last third	`-  ->  -`
Performance worsened	`no`
Worst-drifting feature	`-`
Peak mean PSI	`5.782`

Drift verdict

Frozen-model RMSE held up across time (109.947 -> 99.847; drift magnitude -9.774). Peak input drift (mean PSI) = 5.782 (notable). A sliding-retrained model beat the frozen one (RMSE 90.258 vs 93.723, gap 3.465 and the difference is statistically significant).

Statistical significance

A score difference is only meaningful if it survives a significance test. Paired predictions on the same held-out rows are compared with the Wilcoxon signed-rank test. A p-value below 0.05 means the difference is unlikely to be chance; above it, the two models are statistically indistinguishable - which is itself worth reporting.

comparison	better model	p-value	conclusion
champion vs runner up	linear	`0.1296`	`not significant`
static vs adaptive	adaptive	`3.919e-101`	`significant`

champion vs runner up: linear has lower absolute error; but not significantly (p=0.1296).

static vs adaptive: adaptive has lower absolute error; the difference is statistically significant (p=3.919e-101).

APPENDIX A

Scope and limitations

Tuning is measured, not assumed

The point of Stage 3 is to quantify the cost/benefit of tuning, not to prove that tuning always wins. On well-conditioned datasets a regularized baseline can match a tuned booster, and this report says so when it happens.

Drift is detected, not explained

PSI and prequential error are cheap, reliable indicators that a distribution has moved. Neither attributes a cause; deeper causal drift analysis is out of scope.

Single-machine scale

The pipeline targets datasets that fit in memory on one machine. It is not a distributed training system.

One split, one seed

Results come from a single temporal split at a fixed random seed. That is the right choice for a time-ordered problem - shuffled k-fold would leak the future - but it means confidence intervals across resamples are not reported, which is why the significance tests above matter.

APPENDIX B

Glossary

ROC-AUC	Probability a random positive is ranked above a random negative. 0.5 is chance, 1.0 is perfect. Insensitive to class imbalance.
RMSE / MAE	Average prediction error in the target's own units. Lower is better; RMSE punishes large misses harder than MAE.
R-squared	Share of the target's variance the model explains. 0 means no better than predicting the mean.
Regularization (L1/L2)	A penalty on coefficient size that trades a little bias for less variance. L1 can zero coefficients outright, performing feature selection; L2 shrinks them smoothly.
Stacking	Training a small meta-model on the base models' predictions so it can learn which to trust where.
Bayesian optimization	Tuning that builds a surrogate model of the search space to choose each next trial, rather than sampling blindly.
SHAP value	A feature's signed contribution to one prediction, derived from cooperative game theory. Contributions sum to the prediction.
PSI	Population Stability Index: how far a feature's distribution has moved from its training reference. Rules of thumb: below 0.1 stable, 0.1-0.25 moderate, above 0.25 material.
Prequential evaluation	Test-then-train: each chunk is predicted before the model is allowed to learn from it, so no future information leaks.
McNemar's test	A paired test for two classifiers on identical rows, based only on the cases where they disagree.

Produced by the TabML-Guard pipeline.

~ Ayush Gole